

■ Condition

expr /; *test* gives *expr* if *test* is True.

You can use /; to control the applicability of rules.

Example: `f[x_] := fp[x] /; x > 1` defines a function in the case when $x > 1$. ■ `lhs := rhs /; test` defines *lhs* to be transformed to *rhs* so long as *test* is True. ■ `lhs -> rhs /; test` is a rule that represents replacement of *lhs* by *rhs*, so long as *test* is True. ■ `lhs = rhs /; test` and `lhs -> rhs /; test` are not usually useful, since the `Condition` in these cases is evaluated at the time when the transformation rule is given, not at the time when it is used. ■ `Condition` returns the value `Null` when the test is not True. ■ `Condition` does not evaluate its first argument when the test does not yield True. ■ See page 229. ■ See also: `If`, `Switch`, `Which`, `PatternTest`.